
Software Design Specifications

for

Fifa world cup Data Analytics & Visualization platform

Prepared by:

B. Charan Tej(se23ucse240)
Student

Adrija bandi(se23uari004)
student

saketh krishna rao(se23uari013)
student

kushal(se23uari073)
student

Nithya Rao(se23uari125)
Student

Meka Lakshmi Amrutha
(se23ucse110)
student

sameehan
Sriharsha(se23uari069)
student

Version history:

Version no.	Version date.	Revised by	Description	File Name
1	08-04-2026	Charan Tej	This is the initial version of document.	Software Design Specification

Table of Contents

1 INTRODUCTION	4
1.1 PURPOSE	4
1.2 SCOPE	4
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	4
1.4 REFERENCES	4
2 USE CASE VIEW	4
2.1 USE CASE	4
3 DESIGN OVERVIEW	4
3.1 DESIGN GOALS AND CONSTRAINTS	5
3.2 DESIGN ASSUMPTIONS	5
3.3 SIGNIFICANT DESIGN PACKAGES	5
3.4 DEPENDENT EXTERNAL INTERFACES	5
3.5 IMPLEMENTED APPLICATION EXTERNAL INTERFACES	5
4 LOGICAL VIEW	5
4.1 DESIGN MODEL	6
4.2 USE CASE REALIZATION	6
5 DATA VIEW	6
5.1 DOMAIN MODEL	6
5.2 DATA MODEL (PERSISTENT DATA VIEW).....	6
5.2.1 Data Dictionary.....	6
6 EXCEPTION HANDLING	6
7 CONFIGURABLE PARAMETERS	6
8 QUALITY OF SERVICE	7
8.1 AVAILABILITY.....	7
8.2 SECURITY AND AUTHORIZATION	7
8.3 LOAD AND PERFORMANCE IMPLICATIONS	7
8.4 MONITORING AND CONTROL	7

1 Introduction

1.1 Purpose

This Software Design Specification (SDS) document provides a detailed description of the design and architecture of the FIFA World Cup Data Analytics & Visualization Platform.

It is intended for:

- *Developers for system implementation*
- *Testers for validation*
- *Stakeholders for design understanding*

1.2 Scope

This document describes the design of a web-based system that:

- Processes FIFA World Cup datasets
- Performs analytics and KPI computation
- Provides REST API services
- Displays interactive visual dashboards

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
KPI	Key Performance Indicator
REST	Representational State Transfer
UI	User Interface
DB	Database

1.4 References

- Software Requirements Specification (SRS)
- FastAPI Documentation
- PostgreSQL Documentation
- ReactJS Documentation

2 Use Case View

2.1 Use Case 1: View Team Statistics

Description:

Allows users to view statistical performance of football teams including win rate and goal difference.

Actors:

- Analyst
- Fan/User
- Researcher

Flow of Events:

1. User selects "Team Statistics"
2. System requests data from backend API
3. Backend retrieves data from database
4. Data is returned and displayed

2.2 Use Case 2: View Match History

Description:

Allows users to explore historical FIFA match data.

Flow of Events:

1. User selects “Match History”
2. System sends API request
3. Backend retrieves match data
4. Data displayed in UI

2.3 Use Case 3: View KPI Dashboard

Description:

Displays analytics insights such as trends and performance metrics.

Flow of Events:

1. User opens dashboard
2. System fetches KPI data
3. Data is visualized using charts

3 Design Overview

3.1 Design Goals and Constraints

Design Goals

- Modular and scalable architecture
- High performance and low latency
- Interactive and responsive UI
- Efficient data processing

Constraints

- Backend: FastAPI
- Database: PostgreSQL
- Cache: Redis
- Frontend: React.js
- Data Processing: Python (Pandas, NumPy)

3.2 Design Assumptions

- FIFA datasets are available
- Users access system through browsers
- Network connectivity is stable

3.3 Significant Design Packages

1. Data Processing Layer

- Data ingestion
- Data cleaning
- Data validation

2. Analytics Layer

- KPI computation
- Statistical analysis

3. Backend Layer

- REST API implementation
- Business logic

4. Database Layer

- PostgreSQL storage
- Redis caching

5. Frontend Layer

- UI rendering
- Data visualization

3.4 Dependent External Interfaces

External Application	Interface	Description
Kaggle Dataset	CSV Input	Data source
Web Browser	HTTP	User interaction
Redis	Cache API	Data caching

3.5 Implemented Application External Interfaces (and SOA web services)

Interface Name	Module	Description
/api/teams/stats	Backend	Returns team statistics
/api/matches/history	Backend	Returns match history
/api/kpis/{year}	Backend	Returns KPI data

4 Logical View

4.1 Design Model

Modules

1. DataProcessor

- Handles cleaning and validation

2. AnalyticsEngine

- Computes KPIs and metrics

3. APIController

- Manages REST endpoints

4. DatabaseManager

- Handles database operations

5. CacheManager

- Manages Redis caching

Class Components

- Team
- Match
- Player
- KPI
- User

4.2 Use Case Realization

Use Case: View KPI Dashboard

Sequence Flow:

1. User → Frontend
2. Frontend → API (/kpis/{year})
3. API → Database/Cache
4. Data returned to frontend
5. Visualization rendered

5 Data View

5.1 Domain Model

Entities:

- Team
- Match
- Player
- KPI

Relationships:

- Team participates in Match
- Match includes Players
- KPI derived from Match data

5.2 Data Model (persistent data view)

Tables

matches_table

- match_id
- team1
- team2
- score
- match_date

teams_stats_table

- team_id
- wins
- goals
- matches_played

players_table

- player_id
- name
- team

kpi_analytics_table

- year
- win_rate
- goal_difference

5.2.1 Data Dictionary

Field	Description
team_id	Unique identifier
match_date	Date of match
winner	Winning team

6 Exception Handling

Exception	Handling
No Data Found	Display message
API Failure	Return HTTP error
Database Error	Retry & log
Invalid Input	Validation error

7 Configurable Parameters

Parameter	Usage	Dynamic
DB_URL	Database connection	No
CACHE_TTL	Cache expiry	Yes
API_RATE_LIMIT	Request control	Yes

8 Quality of Service

8.1 Availability

- High system availability
- Database backup support

8.2 Security and Authorization

- HTTPS communication
- Input validation
- Protection against SQL injection
- API rate limiting

8.3 Load and Performance Implications

- API response < 500 ms
- Supports concurrent users
- Redis caching for performance

8.4 Monitoring and Control

- API logging
- Error tracking
- Performance monitoring